

Private Airdrop

Wallet-Bound Membership Proofs for Relayed Rewards

Technical report

Hachem Nasri

Independent project · Kasserine, Tunisia

phigor.arc@gmail.com

Abstract

A public reward claim can reveal a connection between the wallet that qualified and the address receiving the payment. Private Airdrop explores whether that connection can be kept out of the claim transaction. Participants register off chain, generate a membership proof in their browser, and ask a relayer to submit it. The contract verifies eligibility and pays a chosen recipient without receiving the original wallet as a public proof input.

This technical report presents the protocol, its implementation and an evaluation using circuit, helper and contract tests. The results support agreement between the browser's input preparation, the proof and the contract's acceptance rules. They also expose an important limit: the current tier issuer sees information that lets it link a registration request to a later claim. The prototype therefore reduces public disclosure, but does not provide anonymity from its operator or prevent multiple registrations by one person. Its contribution is a working, inspectable integration and a clear account of these boundaries, rather than a new cryptographic construction.

Keywords: private rewards; zero-knowledge proofs; Merkle membership; Ethereum; nullifiers; relayers.

Contents

1	Problem	2
2	Threat model	3
3	Design goals	3
4	Protocol	4
5	Implementation	7
6	Security and privacy properties	9
7	Experiments and tests	9
8	Limitations	10
9	Possible alternatives	11
10	Future research	12
11	Conclusion	12
	Appendix: exact protocol representation	13
	References	14

1 Problem

Consider a participant who qualifies for an airdrop using an existing wallet but wants to receive the reward at a separate address. If the original wallet submits the claim, the transaction can reveal that relationship. If the recipient submits it, the recipient first needs funds for gas, which may introduce another observable link. Private Airdrop separates three actions that are often combined: registering eligibility, proving it, and submitting the payment transaction. Registration happens on a server. A proof establishes membership without publishing the original wallet or its registration secrets. A relayer submits the claim and pays gas, so the recipient need not send the transaction.

These choices do not solve every privacy problem. The server still observes requests, and a recipient may already be associated with a person. The purpose of the project is to implement a precise claim mechanism and examine where its protection begins and ends.

1.1 Questions and contribution

The investigation follows three questions:

1. **Claim binding:** Can one proof connect wallet ownership, registration membership and a chosen recipient without exposing the membership witness on chain?
2. **Consistency:** Can the off-chain helpers, circuit and contract agree on the same claim despite using different data representations?
3. **Limits:** Which guarantees come from the proof, which require contract state, and which remain dependent on an operator?

The result is an undergraduate systems prototype, evaluated through executable tests and a Sepolia deployment. It combines existing cryptographic mechanisms; it does not introduce a new proof system or claim that private group membership is a new idea. Its value is in the explicit design, working integration and examination of failure cases.

1.2 Why use a blockchain?

A public contract makes the accepted membership root, payout policy and completed payments inspectable. It can reject a repeated claim or a payment that exceeds an airdrop's cap according to published rules.

A conventional server would be simpler if one organization controlled the entire process and users accepted its accounting. Here, the contract provides independently executable checks, while the owner still chooses the eligible set and retains administrative powers. The design is therefore not fully trustless.

1.3 Background and related work

1.3.1 Membership without revealing the member

A Merkle tree combines many leaf hashes into one root by repeatedly hashing pairs. A membership path contains the sibling hash needed at each level. With a leaf, its position and those siblings, a verifier can reconstruct the root.

Publishing that path would reveal which leaf is being used. A zero-knowledge proof instead allows the participant to show that such a reconstruction succeeds without publishing the leaf's private contents or its path. The hidden values are called the **witness**. The values available to the verifier are the **public inputs**.

A proof does not choose which root is legitimate. The application must associate the proof with a root accepted by its own policy. In this project, an owner publishes that root when opening claims.

1.3.2 Preventing repeated claims

Hiding the member creates a second problem: how can the application recognize a repeated claim? A nullifier provides a public marker derived from the participant's private registration values. The contract remembers used markers and rejects a second use within the same airdrop.

Semaphore is related work on anonymous group interactions using membership proofs and nullifiers [1]. Private Airdrop adopts this broad pattern but uses a wallet-bound leaf and its own reward-tier policy. It is not an implementation of Semaphore and does not inherit that protocol's full design or security analysis.

1.3.3 Private payments and observed privacy

Zerocash demonstrates a payment construction that hides payment origins, destinations and amounts using zero-knowledge proofs [2]. ZETH studies adapting that approach to Ethereum's account-based ledger [3]. These works place this project in a broader private-payment setting, but Private Airdrop has a narrower objective: it hides the registration witness while leaving the recipient and payment public. It is neither a shielded currency nor a reimplement of those systems.

Cryptographic protection must also be examined in use. Kappos et al. show that identifiable usage patterns can reduce the anonymity available in deployed Zcash [4]. Béres et al. study Ethereum address linking through transaction patterns and examine leakage around a mixer [5]. These findings motivate examining the whole workflow rather than assuming that a zero-knowledge proof alone provides end-to-end anonymity. This project does not reproduce their experiments or claim their measured results as its own.

1.3.4 Building blocks

The circuit is written in Noir, which expresses the relationships that a valid witness must satisfy [6]. Barretenberg's UltraHonk backend generates the proof and the corresponding Solidity verifier [7]. Wallet ownership uses Ethereum's signed-message format [8].

The reward system derives participant-specific values from tier master keys using HMAC-SHA256 [9]. Contract functions are organized behind a Diamond router [10]. These mechanisms serve different purposes; none replaces the others' checks.

2 Threat model

The protected assets are the participant's registration witness, authorization of the recipient, correct reward accounting and the integrity of administrative keys. The adversaries considered are dishonest claimants, public blockchain observers, a curious or censoring service operator, and administrators who depart from the intended inclusion or reward policy.

The analysis assumes sound underlying cryptographic primitives, matching circuit and verifier artifacts, correct execution of the deployed contracts, and an uncompromised participant browser and wallet. These are assumptions, not properties established by the test suite. Witness secrecy also depends on keeping the registration backup private and choosing adequate secret material.

The public chain exposes the recipient, nullifier, tier key, root, reward and transaction timing. The issuer additionally observes a wallet-authenticated nullifier request; the relayer receives the claim. Because one operator runs both services, no non-collusion assumption separates them. Owners determine the accepted tree and published tiers, while the super owner controls upgrades and withdrawals. The protocol does not protect against misuse of those authorized powers.

Transport metadata, compromised endpoints, service denial, human identity and fair admission are outside the proof's protection. The security and privacy analysis below distinguishes what the circuit proves from what the contract enforces and what remains dependent on these actors.

3 Design goals

1. **Limited public disclosure:** establish membership without publishing the original wallet, signature, secret, seed or Merkle path as proof inputs.
2. **Claim integrity:** bind wallet ownership, membership, the chosen recipient and a consistent nullifier in one proof relation.

3. **Stateful payment checks:** enforce the published reward policy, per-airdrop replay prevention, budget limits and available pool balance in the contract.
4. **Relayed submission:** allow a recipient to receive the reward without first funding and submitting its own claim transaction.
5. **Reproducible integration:** keep helper encodings, circuit inputs and verifier artifacts consistent and test their agreement.

These goals concern the implemented claim mechanism. Operator anonymity, one-person-one-claim, fair inclusion, confidential rewards and guaranteed payment are not claimed design achievements.

4 Protocol

4.1 Participants and services

An **owner** creates an airdrop and determines its accepted membership tree. A **participant** registers and later proves a claim. A **recipient** receives the payment and can differ from the participant's original wallet.

The server has three roles. The **issuer** supplies a participant's tier key. The **participant service** stores leaf hashes. The **relayer** sends claims to the blockchain. They run together in the prototype, but their names distinguish their responsibilities.

Finally, the **proof verifier** is a generated Solidity contract. It checks mathematical proof validity. It is not the relayer, and it does not hold the relayer's private key or pay transaction gas.

4.2 One claim from start to finish

Suppose Alice wants to participate in an airdrop. The following sequence describes the implemented workflow; the next sections explain the reasoning behind it.

1. The owner enters a name, budget and reward tiers. The contract creates the airdrop with only the name and budget. A subsequent signed request saves the tiers privately on the server.
2. Alice chooses a secret and seed. Her browser derives a nullifier hash and requests a tier key from the issuer. The request is authenticated by her wallet.
3. Alice's browser hashes her wallet address, secret, seed and tier key into a leaf. She downloads a registration backup and sends the leaf hash to the participant service. No registration transaction is sent.
4. The owner closes registration, fetches the leaves and builds a complete tree. The owner publishes the root and tier policy on chain, then distributes the tree file to participants.
5. Alice imports her backup and the tree. Her browser finds her leaf and its path. She chooses a recipient and signs the claim message using the original wallet.
6. Her browser generates a proof. The relayer submits it, and the contract checks both the proof and the payment rules before sending ETH to the recipient.

Step	Actor	Action and boundary
1	Owner	Create the airdrop on chain; save tiers privately on the server.
2	Participant	Request a tier key; save the private backup; register a leaf off chain.
3	Owner	Close registration; build and distribute the tree; publish root and tiers on chain.
4	Participant	Import the tree, sign and generate the proof locally.
5	Relayer and contract	Submit the claim; verify proof and payment rules; pay the recipient on chain.

Table 1: Registration and claim lifecycle, ordered from top to bottom.

4.3 The two files

Alice's registration backup contains her private registration information. It must remain private. Losing it can prevent her from reconstructing the witness. The interface requires a download before registration and requires a new download if the registration inputs change. This protects against a common user mistake; it does not prove that the file has been stored safely.

The Merkle tree JSON is a different file. It contains the complete tree needed to locate a leaf and derive its path. The owner must distribute it; claimants upload it rather than asking the database for a wallet-specific path. The service currently stores leaves, not downloadable tree files.

4.4 Registration and reward tiers

4.4.1 Committing the registration values

Let P denote the Pedersen hash used consistently by the helpers and circuit. Let a be the wallet address represented as a field element, s the normalized secret, n the seed and t the participant's tier key. The registration leaf is:

$$L = P([a, s, n, t])$$

The leaf binds these values together. Changing any one of them changes the leaf, so a participant cannot simply replace their tier key or wallet when preparing the later claim.

MongoDB stores only the finalized leaf hash in each participant record. It does not store the secret, seed or wallet as separate registration fields. Duplicate leaves are handled idempotently, and the service limits each airdrop to 256 registered leaves.

This storage choice reduces what is retained in the registration database. It does not make all server activity anonymous: tier requests disclose additional information, discussed in the privacy analysis.

4.4.2 A master key and a participant key

Each tier has a master key, an ETH reward and an optional display name. The issuer keeps the master private during registration. A participant receives a derived child key, which is included in their leaf.

The derivation uses three inputs: the tier master, a fixed airdrop context and the participant's nullifier hash. The nullifier is:

$$N = P([s, n])$$

The context is assigned when the airdrop is created. It incorporates the chain, diamond address and airdrop ID. A fixed context is necessary because the final tree root does not exist during registration. Using that root to derive keys already contained in the leaves would introduce a circular dependency.

The child key is deterministic: the same master, context and nullifier reproduce the same value. Distinct inputs are expected to give distinct values with overwhelming probability, but there is no absolute uniqueness guarantee. These are application keys, not BIP-32 child wallets or signing keys.

4.4.3 How the tier is chosen

The prototype assigns a tier deterministically to the authenticated wallet using a keyed hash. Repeated requests from that wallet retain the same selection while the configuration stays fixed. A separate derivation then creates the child key for the requested nullifier.

This is a demonstration allocation rule, not verifiable randomness. A person can create more wallets to sample additional assignments. The request returns the key without its associated reward; that interface choice is not a cryptographic promise to keep the reward secret.

4.4.4 Publishing the policy

The tiers entered at creation are saved off chain after the creation transaction confirms. These steps are not atomic. If saving fails, the interface retains the confirmed airdrop and offers a retry rather than creating another one.

When claims open, the owner publishes the masters and rewards alongside the root. Solidity can then recompute each possible child key and identify the matching reward. Publishing a master does not by itself create a new eligible leaf: the claim must still prove membership in the already accepted root.

The chain does not commit to the private tier configuration at creation. An owner who publishes different masters or rewards can therefore invalidate the intended policy. Loading the original settings in the UI helps operational consistency, but is not a cryptographic restriction on an owner using the contract directly.

4.5 The tree and the proof

4.5.1 Completing the tree

The circuit uses eight levels, giving 256 leaf positions. The helper preserves the fetched leaf order and fills unused positions with deterministic, circuit-shaped leaf hashes. Their values contribute to the root exactly like all other leaves.

Padding makes the tree complete; it does not create additional users. An airdrop with one genuine participant remains a one-participant airdrop for privacy analysis. The design does not count filler leaves as an anonymity set or claim a formal proof of their unspendability.

At each level, an index bit determines whether the current node is hashed on the left or right of its sibling. Pair order is therefore significant. Eight successive hashes reconstruct the root.

When a claimant imports a tree, the helper checks its shape and recomputes its levels rather than trusting arbitrary path data. It then finds the locally computed leaf and extracts its eight siblings. Searching and validating a complete file take more work than the eight-step path check inside the circuit.

4.5.2 What the verifier sees

The proof has four public inputs. Everything needed to demonstrate their relationship belongs to the private witness.

Visibility	Values
Public	Child tier key; nullifier hash; accepted Merkle root; recipient address.
Private	Wallet public-key coordinates; signature; secret; seed; Merkle siblings; leaf index.

“Private” means hidden by the proof from the public verifier. The browser still handles these values to generate the proof. A compromised browser or malicious frontend is outside that protection.

4.5.3 The three relationships checked by Noir

First, wallet ownership. The participant signs a message formed from the recipient and seed. The circuit verifies that signature, derives an Ethereum address from the supplied public key, and uses that address in the leaf. Knowing another person’s wallet address is therefore insufficient.

Second, membership. The circuit recomputes the leaf and follows the private path to a root. It requires that root to equal the public root. The Solidity caller supplies the root stored for the selected airdrop.

Third, nullifier consistency. The circuit recomputes the nullifier from the same secret and seed used in the leaf. This prevents the participant from assigning an unrelated replay marker to an otherwise valid witness.

The membership check is expressed directly in Noir:

```

let computed_leaf = pedersen_hash([
    pub_key_field, secret, nullifier_seed, tier_key
]);
let computed_root = compute_merkle_root(
    computed_leaf, merkle_path, leaf_index
);
assert(computed_root == merkle_root);

```

Here, `let` introduces a value, square brackets preserve the input order, and `assert` requires equality. This is a constraint on a valid witness, not an instruction to replace the stored root. The syntax resembles Rust, but this code belongs to the Noir circuit.

4.5.4 Why byte agreement matters

The wallet signs 32 raw bytes. Signing the printed hexadecimal text of those bytes produces a signature for a different message. Similarly, an address is converted from its 20 bytes to an integer, not hashed as text.

The helpers handle these conversions before witness generation. They also recover the wallet public key from its signature, verify its address and provide the signature format expected by Noir. The browser never needs the wallet's private signing key.

Exact field, signature and message formats are collected in the appendix. Keeping them explicit is necessary for compatibility, but the protocol's central statement remains simple: a valid signer knows a registered leaf whose recipient authorization and nullifier are consistent.

5 Implementation

5.1 Proof validity is not payment authorization

A valid proof alone is insufficient for payment. It cannot know whether a nullifier has already been used or whether the pool still contains ETH. The claim function supplies those stateful checks in this order:

1. Require the configured relayer, an existing airdrop and an unpaused claim phase.
2. Reject invalid field values, a zero recipient and a previously used nullifier.
3. Match the child key against the published tier masters and select its reward.
4. Check the remaining airdrop budget and pool balance.
5. Verify the proof against the child key, nullifier, stored root and recipient.
6. Mark the nullifier used, update the claimed total and send the reward.

If the recipient rejects the payment, the whole transaction reverts, including the nullifier update and event. A reentrancy lock protects external calls. State changes before payment and the lock are complementary protections.

The nullifier mapping belongs to each airdrop. It prevents repetition there, not across every deployment or application. A participant can also register a different leaf with a different nullifier; the rule is not one-wallet-one-claim.

5.2 Lifecycle and funding

The contract has three phases: Registration, Claim and Ended. Paused is a separate flag. Preparing the tree freezes registration on the server before the owner opens Claim on chain; this is an operational step, not a fourth contract phase. An ended airdrop cannot be reopened by the ordinary interface.

All rewards come from a shared ETH pool. An airdrop budget is a spending cap, not a reserved deposit. Several caps can exceed the total funds available. A valid claim may consequently fail because another airdrop consumed the pool or because the super owner withdrew funds.

The relayer wallet needs its own gas balance. Depositing ETH into the pool does not fund the relayer, and the prototype does not deduct gas fees from rewards.

5.3 Two facets behind one address

The diamond presents one application address and routes calls by their four-byte function selectors. The admin facet implements management operations, while the user facet handles claims and public queries. Both use diamond storage through delegated execution.

The verifier remains a separate contract called during claims. The role faucet is also separate: it asks the diamond to grant demo access. Neither is an additional application facet.

Component	Responsibility and authority
Diamond and shared storage	Application address; selector routing and shared state; six immutable diamond functions.
AdminFacet	Ten routed selectors for administrative operations.
UserFacet	Eleven routed selectors for claims and public queries; calls the proof verifier.
Generated verifier	Separate contract that checks mathematical proof validity.
DemoFaucet	Separate contract that requests demo roles from the diamond.
Relayer wallet	External authorized sender that submits claims and pays gas.
Super owner	Controls upgrades, withdrawals and access settings.

Table 2: Contract structure and authority. Facets execute through the diamond; the verifier and faucet are separate contracts.

This organization separates responsibilities without claiming a throughput advantage. It also introduces upgrade trust: the super owner can replace routed implementations. The fixed address should not be mistaken for immutable application policy.

5.4 Roles and review access

Owners can create and manage all airdrops, not only their own. The original deployer retains the super-owner role, which controls permanent role assignment, the authorized relayer, faucet settings, upgrades and withdrawals.

The demo faucet allows a reviewer to obtain ordinary owner powers while it is enabled. Disabling or replacing it removes the effectiveness of its demo grants, but does not undo prior actions. Re-enabling the same faucet can reactivate its earlier grants. Permanent owners are separate.

Read-only review uses the same admin screens with writes disabled. Demo owners see those screens with their permitted actions available; they do not gain super-owner powers. These modes improve inspectability, but UI restrictions are not the authorization boundary. The contracts and server check roles independently.

5.5 Services and artifact consistency

The issuer authenticates signed requests tied to the chain, diamond, airdrop, action, payload and expiry. Owner requests additionally require a current on-chain owner role. Tier settings are kept in private server files, separately from MongoDB leaf records.

The relayer validates a submitted claim, simulates the actual contract call, then broadcasts from its configured wallet. Requests are queued to avoid wallet nonce races. Simulation reduces avoidable failures but cannot reserve state: the airdrop or balance may change before mining.

The browser and server import generated contract ABIs. The browser's circuit artifact and Solidity verifier are generated from the same circuit using matching backend settings. This addresses a common integration failure: inputs and proofs that are internally valid but belong to different circuit/verifier versions.

6 Security and privacy properties

6.1 Protection from a public observer

The claim does not expose the original wallet public key, signature, secret, seed or membership path as public inputs. Under the proof system's assumptions, the proof establishes the required relationship without revealing that witness. The relay also removes the need for the recipient to fund and send its own claim transaction.

This does not ensure that the recipient is anonymous. The address may already have a public identity, later transfers may reveal connections, and timing or a very small participant set may support inference. Empirical privacy studies make this distinction between hidden transaction data and observable usage patterns especially relevant [4], [5].

The interface rejects using the connected wallet as recipient, but the circuit does not enforce an inequality between those addresses. That restriction helps the intended workflow without becoming a proof-level guarantee.

Question	Enforced mechanism	Outside that mechanism
Is this a member?	Circuit reconstructs the accepted root.	Fairness of registration and owner inclusion.
Does the signer own the committed wallet?	Signature verification and address-derived leaf.	Compromised wallet or frontend.
Has this claim been used?	Per-airdrop nullifier mapping.	One-person-one-claim and cross-airdrop privacy.
Is this the right reward?	Contract recomputes child keys from published masters.	Honesty of the policy chosen by the owner.
Will the reward be paid?	Budget, balance and transfer checks.	Funding, relay availability and trusted administration.

7 Experiments and tests

7.1 Method

Evaluation separates three layers: input preparation, circuit constraints and contract policy. Noir tests receive fixtures generated by the same Rust helpers compiled for the browser. Independent hash vectors add a check beyond comparing two calls to the same function.

Most Solidity application tests use a verifier double so that policy failures can be isolated. A separate suite uses the actual generated verifier and a real proof. Passing a policy test with a double is not counted as end-to-end cryptographic evidence.

The recorded environment used Noir 1.0.0-beta.26, Barretenberg 5.2.0 and Solidity 0.8.30 with optimization enabled. The following are passing test counts, not source coverage percentages.

Suite	Passed	What it exercises
Noir circuit	60	Wallet ownership, membership, nullifier and public-input consistency; invalid witnesses and circuit boundaries.
Rust helpers	6	Normalization, exact phrase bytes, tree padding/import and tier derivation.

Suite	Passed	What it exercises
Solidity policy	20	Roles, lifecycle, budgets, transfers, nullifier reuse, demo access and routing. Includes a 256-run budget fuzz test.
Real verifier	3	A valid proof, altered public inputs and damaged proof bytes.

7.2 Findings from the tests

Membership tests cover every position in a 256-leaf tree and change sibling values, indices and registration inputs. The all-position check is one test that iterates over the tree, not 256 separate named tests. Ownership tests reject malformed or inappropriate signatures and keys.

Normalization tests matter because a correct mathematical design can fail through inconsistent bytes. They check field ranges, address interpretation and exact phrase encoding. Tree tests validate imported levels and deterministic padding. Tier vectors compare the cross-language derivation.

Several cases deliberately document circuit boundaries. A proof for a self-created tree can satisfy Noir but is not a claim against an unapproved root. Replay satisfies the stateless relation again but must fail against consumed contract state. The circuit does not check master-key derivation or restrict a recipient field to 160 bits; contract integration supplies those constraints.

The three real-verifier tests establish that a generated proof is accepted and that the tested changes to its public inputs or proof bytes are rejected. This supports compatibility between the circuit and generated Solidity, while leaving broader cryptographic security to the proof system's assumptions and analysis.

7.3 Sepolia integration

Deployment completed eight successful transactions, including two verifier library deployments and faucet registration. All seven deployed contract sources were verified on Etherscan. Post-deployment reads confirmed both facet selector sets, the verifier, relayer and super-owner addresses, and the enabled faucet connection.

The deployed verifier accepted the real Noir proof fixture through a read-only call. A role-faucet request also succeeded in simulation. These observations validate wiring and verifier compatibility; they do not establish that a participant completed the full funded claim workflow on Sepolia.

At that post-deployment check, the new instance contained no airdrops and both the pool and relayer balance were zero. A live claim still requires registration, tree distribution, claim opening and funding. The repository's deployment manifest records the addresses for inspection without making the technical report depend on a long address listing.

7.4 Limits of the evidence

The tests exercise intended behavior and selected failures. They are not an audit, exhaustive input exploration or a proof of security. Explorer verification matches source to deployed code; it does not evaluate the design's safety.

No proof-time, memory or throughput benchmark is claimed. Test gas figures would also be misleading as payment costs when they include fixture setup or use a verifier double. A performance study should measure actual proving and claims under stated hardware, browser and network conditions.

8 Limitations

8.1 The issuer can link a claim

The issuer receives a wallet-authenticated request containing the nullifier hash. That hash later appears publicly in the claim. An operator who observes or records the request can connect the original wallet to the recipient transaction.

This is the most important privacy limitation in the current architecture. Storing only leaf hashes in MongoDB does not remove information available during request processing. Separating the issuer and relay into code modules does not separate their knowledge when one operator runs both.

Stronger operator privacy would need a different issuance mechanism, not simply fewer database columns. The issuer should be unable to recognize a later claim from what it observed while authorizing the participant. Transport metadata and logging would also need separate treatment.

8.2 Replay is different from Sybil resistance

The contract rejects a repeated nullifier within an airdrop. It does not establish that each leaf belongs to a different person. Multiple wallets, or multiple nullifiers from one wallet, can produce multiple registrations. Moreover, the registration endpoint accepts a valid field-valued leaf without proving its ownership or tier entitlement at admission. Arbitrary leaves can occupy the limited capacity even when no successful claim can be constructed for them. A production admission policy would need to address both repeated participation and capacity abuse.

The nullifier hash contains only the secret and seed. Reusing both across airdrops produces the same public marker, making claims linkable despite separate spent mappings. Fresh inputs avoid that particular link; explicit application scoping inside the nullifier would be a protocol change.

8.3 Owner and availability trust

The owner can omit an eligible leaf or publish mismatched tiers. The circuit proves membership in the accepted set, not that the owner assembled the set fairly. The design has no forced-inclusion or dispute mechanism.

The super owner can withdraw pool funds and change code. The relay can decline to submit a valid claim. These are real trust and availability dependencies, even when all cryptographic checks work correctly.

Published masters and rewards also make tier association computable once the public claim values are known. Showing an amount only after confirmation is therefore a UI choice, not reward confidentiality.

8.4 Capacity and policy commitments

The fixed-depth tree keeps the proof interface and tests manageable, but limits one airdrop to 256 leaves. Its padding gives a consistent shape rather than meaningful cover traffic. Increasing capacity would require evaluating the circuit, file distribution and user experience together.

Publishing tier masters makes reward verification straightforward after registration closes. The accepted root then prevents newly derived keys from becoming eligible by themselves. However, the protocol still trusts the owner to publish the originally intended policy. An earlier commitment could reduce that trust at the cost of another protocol step.

9 Possible alternatives

The following are design options, not implemented or experimentally compared alternatives.

1. **Server-managed rewards:** a conventional service could retain registration and payment accounting, simplifying the architecture while requiring participants to trust its records and execution.
2. **An earlier policy commitment:** committing to tier settings at creation could let participants check the subsequently published policy. This would require commitment and opening rules and would not by itself ensure inclusion or funding.
3. **Unlinkable issuance:** an issuance mechanism that hides the later claim identifier from the issuer could address the current operator-linkage channel. It would require a separate construction and security analysis, including how eligibility and reward assignment remain enforceable.
4. **Scoped nullifiers:** including application context in nullifier derivation could address the repeated marker caused by reusing a secret and seed across airdrops. This would change the circuit and associated artifacts and would not provide human uniqueness.

10 Future research

The most important next step for privacy is unlinkable issuance. The most important next step for abuse resistance is a clearly defined admission policy. They address different adversaries and should be evaluated separately; adding one should not be presented as solving the other.

Further work should also measure browser proving on ordinary devices, test concurrent service operation, and examine recovery when a backup or tree file is unavailable. Those studies would turn an inspectable prototype into stronger evidence about practical deployment.

11 Conclusion

Private Airdrop demonstrates a wallet-bound membership proof combined with relayed ETH rewards. Its tested integration connects registration values, a Merkle root, a recipient authorization and a nullifier, while the contract enforces the stateful payment rules.

The distinction between these layers is the main result. A valid proof can establish the required relation without establishing fair admission, human uniqueness, funding or operator anonymity. Making those limits explicit provides a credible basis for improving the protocol rather than attributing guarantees to zero knowledge that the complete system does not provide.

Appendix: exact protocol representation

This appendix collects the implementation details needed to interpret or independently implement the statement. The preceding sections describe their purpose; this section fixes their representation.

Field and text conversion

The field modulus is:

```
21888242871839275222246405745257275088548364400416034343698204186575808495617
```

Numeric inputs must be between zero and one less than this value. Rust accepts decimal or explicitly prefixed hexadecimal numeric strings, then returns canonical decimal strings. It rejects oversized values instead of silently reducing them. Server field inputs use canonical decimal strings. This avoids precision loss through JavaScript numbers.

A wallet or recipient address is exactly 20 bytes interpreted as a big-endian integer. The secret phrase is handled differently: its exact UTF-8 bytes are Keccak-hashed and reduced modulo the field. There is no trimming, case folding or Unicode normalization. This is not a password-hardening function; users should not rely on a weak phrase as their only source of secrecy.

The seed and tier key are numeric fields. Rewards are integer wei, with 18 decimal places per ETH, and do not pass through field or phrase normalization.

Child-key derivation

With BE32 meaning exactly 32 big-endian bytes and `||` meaning concatenation:

```
context = Keccak256(ABI.encode(chainId, diamond, airdropId)) mod p
N = Pedersen([secret, seed])
t = HMAC-SHA256(
    key = BE32(master),
    message = UTF8("private-airdrop/tier-key/v1")
                || BE32(context) || BE32(N)
) mod p
```

The issuer's tier-choice hash is separate: it uses the first tier master as its HMAC key and deployment scope, airdrop ID and lowercase wallet as its message, then reduces modulo the number of tiers. Solidity checks the child derivation against every published tier until it finds a match. At most 32 tiers are accepted.

Message and signature

The signed message and digest are:

```
M = BE32(Pedersen([recipient, seed]))
D = Keccak256("\x19Ethereum Signed Message:\n32" || M)
```

The prefix is 28 bytes and M is 32 bytes. The circuit verifies an ECDSA signature against D. A normal wallet signature includes r, s and a recovery identifier; the helper uses recovery to obtain the public key and passes the 64 r-and-s bytes to Noir. It accepts the expected recovery encodings and rejects high-s signatures rather than rewriting them.

The wallet address comes from the last 20 bytes of Keccak over `x || y`, where each coordinate is 32 bytes. The uncompressed public key's leading 04 is omitted. No wallet private key is part of the witness.

The signature directly binds the recipient and seed, not the chain, diamond or airdrop ID. The context enters the child derivation and membership relation, while the contract supplies the approved root. This should not be described as direct deployment scoping inside the signed message.

Noir input types

Input	Type	Visibility
pub_key_x, pub_key_y	[u8; 32] each	Private
signature	[u8; 64]	Private
secret, nullifier_seed	Field each	Private
merkle_path	[Field; 8]	Private
leaf_index	Field	Private
tier_key	pub Field	Public, first
nullifier_hash	pub Field	Public, second
merkle_root	pub Field	Public, third
recipient	pub Field	Public, fourth

[u8; 32] denotes 32 unsigned bytes; Field denotes a field element. In a main input declaration, pub exposes a verifier input. By contrast, pub fn concerns source-level function visibility and does not make its arguments public proof inputs.

The index is decomposed into eight little-endian bits. Starting from the leaf, a zero bit hashes [current, sibling]; a one bit hashes [sibling, current]. Pedersen parameters must match the Noir/Barretenberg implementation, including the helper's hash index zero.

Tree and service conventions

There are 256 leaves, 255 internal nodes and nine stored rows including the root. The service returns participant hashes sorted lexicographically by their stored identifiers. The builder preserves that order and fills the remaining positions deterministically using an address based on the index, fixed-offset secret and seed values, and tier key zero. Imported trees are rebuilt and checked before path use.

Server authorization messages bind chain, contract, action, airdrop ID, exact JSON payload hash and expiry. Expiry is limited to ten minutes; it is not a consumed one-time nonce. Idempotent operation semantics handle repeated requests where applicable. Registration and snapshot operations are serialized in one process, not across distributed replicas.

The implementation uses Noir 1.0.0-beta.26, Barretenberg 5.2.0 and matching EVM proof settings. The generated verifier manifest records circuit and verification-key hashes. Changing the circuit requires regenerating its paired artifacts; an ABI update alone does not establish proof compatibility.

References

- [1] Semaphore Contributors, "Semaphore: Anonymous Interactions." [Online]. Available: <https://semaphore.pse.dev/learn>
- [2] E. Ben-Sasson *et al.*, "Zerocash: Decentralized Anonymous Payments from Bitcoin," in *2014 IEEE Symposium on Security and Privacy*, 2014, pp. 459–474. doi: 10.1109/SP.2014.36.
- [3] A. Rondelet and M. Zajac, "ZETH: On Integrating Zerocash on Ethereum." [Online]. Available: <https://arxiv.org/abs/1904.00905>
- [4] G. Kappos, H. Yousaf, M. Maller, and S. Meiklejohn, "An Empirical Analysis of Anonymity in Zcash," in *27th USENIX Security Symposium*, USENIX Association, 2018, pp. 463–477. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity18/presentation/kappos>

- [5] F. Béres, I. A. Seres, A. A. Benczúr, and M. Quinyne-Collins, “Blockchain is Watching You: Profiling and Deanonymizing Ethereum Users.” [Online]. Available: <https://arxiv.org/abs/2005.14051>
- [6] Noir Contributors, “Noir Documentation.” [Online]. Available: <https://noir-lang.org/docs/>
- [7] Aztec Protocol, “Barretenberg.” [Online]. Available: <https://github.com/AztecProtocol/aztec-packages/tree/master/barretenberg>
- [8] M. H. Swende and N. Johnson, “ERC-191: Signed Data Standard.” [Online]. Available: <https://eips.ethereum.org/EIPS/eip-191>
- [9] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-Hashing for Message Authentication,” technical report RFC 2104, 1997. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2104>
- [10] N. Mudge, “ERC-2535: Diamonds, Multi-Facet Proxy.” [Online]. Available: <https://eips.ethereum.org/EIPS/eip-2535>